

Assignment 3

Purpose

For this assignment, you are tasked with analyzing both Bitcoin transactions and consumer sales trends by applying the skills and tools acquired. To complete this assignment, download and import the datasets and then create PostgreSQL queries that respond to each set of corresponding prompts. You will be responsible for testing your code on the provided data set before submission. Each question will be graded based on whether or not it generates the correct output and addresses all requirements specified in the question. **Ensure to only return the columns asked for in each question.**

Submission

You will submit a total of 2 sql files. Files should be named appropriately and be in .sql format. Each file must use the postgres standards taught in the course. Use of other SQL languages such as T-SQL will result in an automatic 0 for the assignment.

- **File 1:** You must submit a SQL document called <LastName>_Assignment3_Part1 to the Assignment 3
- **File 2:** You must submit a SQL document called <LastName>_Assignment3_Part2 to the Assignment 3

Instructions

As in the practice assignments you will be querying two datasets to gather insights about that data. You have been provided SQL files with all the necessary data. Make sure you take a moment to familiarize yourself with the columns, their data types, and any other pertinent information.

Part 1: Bitcoin Data

Data Dictionary

There is a data dictionary for the bitcoin data provided on Brightspace. Note that each row in the bitcoin table contains aggregated transaction information for an entire day.

Initial Steps: Creating the Table and Importing the Data

Note: The code used for this section should not be included in the file you submit. This part is a step by step on how to create the bitcoin table, which you will use to validate your queries.

1. In Github repository, download the **bitcoin_data.csv** file
2. Place this file in a public folder on your computer
3. Copy the path to this file.
4. In github, download the **Bitcoin Table Creation and Importing** sql file. Open this file in pgAdmin (The database you choose to put this in is up to you).
5. Update the code with the path to the bitcoin_data.csv file that you copied in step 3.
6. Run the commands and this should create the table and import the bitcoin data. Table must be named **bitcoin**.

Part 1 Instructions

1. Name this file using the naming convention: <LastName>_Assignment3_Part1
2. Write SQL queries that obtain the requested information based on the prompts in the Queries section below
3. Once you are done, push the document to github

Part 1 Queries

*Please note that column names in **bold** to help you identify the columns from the table to be used in each question. However, these aren't necessarily the columns that your query output should return. Be sure to read each question carefully and return only the columns requested.*

1. Write a query that returns the **trans_date** along with the **code_size** divided by the **transactions**. Show all decimals that would normally result from division and name the calculated column *difficulty*.
2. Write a query that returns the **trans_date** and the product of **median_transaction_fee** and **transactions** for days where the **median_transaction_fee** is more than 0.5. Label the calculated column *daily_cost*.
3. Write a query that returns the **trans_date** and the **sent_in_usd** divided by the number of **transactions** on days where there were more than 400,000 transactions. Show all expected decimals. Name the output column *avg_transaction*.
4. Write a query that returns the average **price_usd** (rounded to 2 decimal places, output as MONEY datatype) and the total amount **sent_in_usd** (output as MONEY datatype) for all transactions that occurred in May of 2015. Name the output columns *avg_price* and *total_transaction_amount*, respectively.

5. Write a query that returns the smallest and largest **market_cap** values as whole numbers and the lowest and highest **price_usd** rounded to 2 decimals. Name these columns as *low_cap*, *high_cap*, *low_price_usd*, and *high_price_usd* respectively.
6. Write a query that finds the lowest **median_transaction_fee**, the highest **median_transaction_fee**, and the average **transaction_fees** for **trans_dates** between August 10, 2017 and August 10, 2019 (inclusive). Round the average to 5 decimal places. Name these columns *lowest_fee*, *highest_fee*, and *avg_fee* respectively.
7. Write a query that determines the average number of **transactions** and average **sent_by_address** for days with more than 350,000 transactions. Name these columns *avg_transactions* and *avg_sent_addresses* respectively.
8. Write a query that shows the average **google_trends** score (as a whole number) and the total number of **tweets**. Name the columns *avg_google_trends* and *total_tweets*.
9. Write a query that returns the minimum, maximum, and average **confirmation_time** for all transactions. Round the average to three decimal places. Name the output columns *min_confirmation_time*, *max_confirmation_time*, and *avg_confirmation_time*.
10. Write a query that returns the averages for the following columns (all rounded to 2 decimals): **price_usd**, **mining_profitability**, and **transaction_fees** for all transactions that happened between March 1, 2020, and November 30, 2020. Name the columns: *avg_price_usd*, *avg_mining_profitability*, and *avg_transaction_fees* respectively.

Part 2: Customer Spending Data

Initial Steps: Creating the Table and Importing the Data

Note: The code used for this section should not be included in the file you submit. This part is a step by step on how to create the customer_spending table, which you will use to validate your queries.

1. From the Github, download the **customer_spending.csv** file
2. Place this file in a public folder on your computer
3. Copy the path to this file.
4. In the Assignment 3 folder, download the **Customer Spending Table Creation and Importing** sql file. Open this file in pgAdmin (The database you choose to put this in is up to you).
5. Update the code with the path to the customer_spending.csv file that you copied in step 3.
6. Run the commands and this should create the table and import the customer spending data. Table must be named **customer_spending**.

Part 2 Instructions

1. Name this file using the naming convention: <LastName>_Assignment3_Part2

2. Write SQL queries that obtain the requested information based on the prompts in the Queries section below.
3. Once you are done, push the Assignment 3 to github

Part 2 Queries

Please note that column names in **bold** to help you identify the columns from the table to be used in each question. However, these aren't necessarily the columns that your query output should return. Be sure to read each question carefully and return only the columns requested.

1. Write a query that returns each **category** and the corresponding total **revenue** for that category for the **sale_year** 2016. The aggregated column should be named *total_revenue*. The output should be arranged alphabetically.
2. Write a query that returns a list of **sub_categories** and their corresponding average **unit_price** (named *avg_unit_price*), average **unit_cost** named *avg_unit_cost*, as well as the difference between these two values (named *margin*) for the **sale_year** 2015. Round all values to two decimal places. Organize the results alphabetically.
3. Write a query that returns the **sale_year** and corresponding total number of female buyers (**gender**) for each **sale_year** who made purchases in the Clothing **category**. Name the aggregated column *total_female_buyers*.
4. Write a query that returns the **age**, **sub_category**, average **quantity** (as a whole number, named *avg_quantity*), and average **cost** of products (rounded to 2 decimals, named *avg_cost*) purchased by each **age** and **sub_category**. Organize the data by **age**, oldest to youngest, and then by **sub_category** alphabetically.
5. Write a query that returns a list of **countries** where more than 900 transactions were made by customers between the **ages** of 18-25 (inclusive).
6. Write a query to identify which **sale_year** and **category** combinations were the most profitable. Return the columns **sale_year**, **category**, the total **revenue** as *total_revenue*, the total **cost** as *total_cost*, and the difference between the two as *profit* for each **sale_year** and **category**. Include only rows that were profitable and sort the results from highest profit to lowest.
7. Write a query that calculates the average spending per **age** for male customers (**gender**), where spending is based on the product of **unit_price** and **quantity**. Return the **age** and the calculated average spending, rounded to two decimal places, as *avg_spending*. Organize the results from highest to lowest *avg_spending*.
8. Write a query to determine the highest **unit_cost**, lowest **unit_cost**, and average **unit_cost** for each **gender** in each **category**. The output columns should be **gender**,

category, *high_cost*, *low_cost*, *avg_cost*, in that order. Organize the results by **gender** and then **category**.

9. Write a query to determine the age distribution of customers by **category** and **country** for the **sale_year** 2016. For each **category** and **country**, calculate the **age** of the youngest and oldest customers, and the average **age** of customers rounded to one decimal place. The output columns should be **category**, **country**, *youngest_customer*, *oldest_customer*, and *avg_customer_age*, in that order. Organize your results by **category** and then by the average age of customers.
10. Write a query to return the **country** that has the highest average **revenue** (rounded to 2 decimals). Your output columns should be **country** and *high_sales*.

Hint: To obtain the highest value, you will need to return only one row from your results. Think of how you'll need to sort the output.